

ROADMAP TÉCNICO · PARA EXECUÇÃO POR IA, FASE A FASE

MedFlow

Sistema de Gestão de Clínica Médica (fictício) —
roadmap completo de engenharia, do zero ao deploy:
backend Java/Spring, frontend React, banco de dados,
testes, Docker, Kubernetes, CI/CD e observabilidade.

Java 21 + Spring Boot

PostgreSQL

React

Docker & Docker Compose

Kubernetes

GitHub Actions (CI/CD)

Prometheus + Grafana

JWT + Spring Security

Sumário

VISÃO GERAL

Sobre este documento e como usá-lo com uma IA	03
O projeto: MedFlow	04
Stack tecnológica completa	05
Arquitetura geral do sistema	06

FASES DE EXECUÇÃO

Fase 0 — Preparação do ambiente	07
Fase 1 — Modelagem e planejamento	09
Fase 2 — Esqueleto do backend	11
Fase 3 — Banco de dados e migrations	13
Fase 4 — Autenticação e autorização (JWT)	15
Fase 5 — Módulo de usuários e perfis	17
Fase 6 — Módulo de médicos e agenda	18
Fase 7 — Módulo de consultas (agendamento)	19
Fase 8 — Módulo de pagamentos fictícios	21
Fase 9 — Prontuário e exames (upload de arquivos)	22
Fase 10 — Notificações assíncronas	23
Fase 11 — Testes automatizados	25
Fase 12 — Documentação da API	27
Fase 13 — Frontend em React	28
Fase 14 — Containerização com Docker	31
Fase 15 — CI/CD com GitHub Actions	33
Fase 16 — Orquestração com Kubernetes	35
Fase 17 — Observabilidade	38
Fase 18 — Segurança e hardening	40
Fase 19 — Deploy em nuvem (opcional)	41
Fase 20 — Extras / próximos níveis	42

ANEXOS

Modelo de prompt para entregar cada fase à IA	43
Checklist geral de entrega	44

Sobre este documento e como usá-lo

Este roadmap foi desenhado para ser entregue a uma IA de codificação (Claude Code, Cursor, etc.), fase por fase.

Este documento é dividido em **fases sequenciais**. Cada fase tem: objetivo, o que precisa ser entregue, quais tecnologias entram naquele momento (e não antes), e um **"Definição de Pronto" (DoD)** — os critérios que confirmam que aquela fase está realmente completa antes de avançar para a próxima.

 **Como entregar isso a uma IA**

Não cole o documento inteiro de uma vez para a IA implementar tudo de uma tacada — ela vai perder contexto e misturar decisões. O ideal é: (1) colar a seção "O projeto" + "Stack tecnológica" + "Arquitetura geral" uma única vez, como contexto fixo do projeto; depois (2) colar **uma fase por vez**, pedindo para ela implementar só aquilo, revisar o resultado, testar localmente, e só então avançar para a próxima fase. No Anexo A há um modelo de prompt pronto para isso.

 **Regra de ouro**

Nunca deixe a IA pular fases "para adiantar". Kubernetes só faz sentido depois que Docker já funciona. CI/CD só faz sentido depois que os testes já existem. A ordem deste roadmap existe justamente para evitar que você acabe com um projeto grande e quebrado sem entender por quê.

O que você vai aprender/treinar em cada etapa do caminho

Etapa do roadmap	O que você treina
Fases 0 a 3	Estrutura de um projeto backend real, camadas, banco de dados versionado
Fases 4 a 10	Regras de negócio, autenticação, múltiplos módulos interligados
Fase 11 a 12	Qualidade: testes automatizados e documentação de API
Fase 13	Integração real front-end/back-end
Fases 14 a 17	"Derrubar e levantar": containers, orquestração, pipelines, monitoramento
Fases 18 a 20	Nível produção: segurança, nuvem, escalabilidade

O projeto: MedFlow

Contexto de negócio fictício, completo, para dar realismo às decisões técnicas.

MedFlow é um sistema de gestão para uma clínica médica fictícia com múltiplas especialidades. O sistema precisa permitir que pacientes agendem consultas, médicos gerenciem sua agenda, a recepção controle check-in e pagamentos, e a administração acompanhe indicadores da clínica.

Perfis de usuário (papéis / roles)

Perfil	O que pode fazer
PACIENTE	Criar conta, ver médicos e especialidades disponíveis, agendar/cancelar suas próprias consultas, ver seu histórico e pagamentos
MEDICO	Ver sua agenda, confirmar/recusar consultas, registrar observações de atendimento (prontuário), definir horários disponíveis
RECEPCIONISTA	Cadastrar pacientes, fazer check-in de consultas, registrar pagamentos, remarcar consultas
ADMIN	Gerenciar médicos, especialidades, ver relatórios/dashboard, gerenciar usuários e permissões

Módulos funcionais do sistema

1. **Usuários e autenticação** — cadastro, login, papéis (roles), recuperação de senha.
2. **Médicos e especialidades** — cadastro de médicos, especialidades, horários de atendimento.
3. **Agendamento de consultas** — o núcleo do sistema: marcar, remarcar, cancelar consultas, com validação de conflito de horário.
4. **Pagamentos (fictícios)** — geração de cobrança por consulta, status (pendente/pago/estornado), sem gateway real — é um "simulador" de pagamento.
5. **Prontuário e exames** — registro de observações médicas por consulta, upload de arquivos de exames (PDF/imagem).
6. **Notificações** — envio assíncrono de "e-mails" (podem ser simulados/logados) de confirmação, lembrete e cancelamento.
7. **Dashboard/relatórios (admin)** — número de consultas por período, faturamento fictício, taxa de cancelamento, médicos mais demandados.

🎯 Por que esse projeto é bom para treinar

Ele tem regra de negócio de verdade (conflito de agenda), múltiplos perfis com permissões diferentes, um fluxo de "dinheiro" (mesmo que fictício), dados sensíveis (força você a pensar em segurança), e módulos que se relacionam entre si — exatamente a complexidade mínima de um sistema real de empresa, sem ser grande demais para travar o aprendizado.

VISÃO GERAL

Stack tecnológica completa

Toda a tecnologia usada no projeto, fixada desde já para a IA não improvisar escolhas divergentes entre fases.

Camada	Tecnologia	Motivo da escolha
Linguagem backend	Java 21 (LTS)	Versão estável mais recente com suporte de longo prazo
Framework backend	Spring Boot 3.x	Padrão de mercado para APIs Java corporativas
Build tool	Maven	Mais previsível para IA gerar configuração corretamente
Banco de dados	PostgreSQL 16	Banco relacional robusto, gratuito, usado amplamente em produção
Migrations de banco	Flyway	Versionamento do schema do banco, junto com o código
ORM	Spring Data JPA + Hibernate	Padrão de mercado em Java
Autenticação	Spring Security + JWT	Autenticação stateless, adequada para API REST
Cache (fase avançada)	Redis	Cache de consultas frequentes (ex.: lista de médicos/especialidades)
Mensageria assíncrona	RabbitMQ	Fila para notificações assíncronas (e-mails simulados)
Documentação de API	springdoc-openapi (Swagger UI)	Documentação interativa gerada automaticamente
Testes	JUnit 5, Mockito, Testcontainers	Testes unitários e de integração com banco real em container
Frontend	React 18 + Vite + TypeScript	Stack moderna, rápida de configurar, tipada
Estilização frontend	Tailwind CSS	Produtividade e consistência visual rápida
Requisições HTTP (frontend)	Axios + React Query	Gerenciamento de chamadas à API e cache de estado do servidor
Containerização	Docker + Docker Compose	Ambiente local reproduzível, multi-serviço
Orquestração	Kubernetes (via Minikube localmente)	Simular ambiente de produção real, escalável
Gerenciador de manifests K8s (opcional avançado)	Helm	Empacotar configuração do Kubernetes de forma reutilizável
CI/CD	GitHub Actions	Pipeline gratuito, integrado ao repositório
Observabilidade	Spring Boot Actuator + Prometheus + Grafana	Métricas e health checks, padrão de mercado
Logs	SLF4J + Logback (estruturado em JSON)	Padrão Java, pronto para ferramentas de agregação de log
Controle de versão	Git + GitHub	Versionamento e histórico de todo o projeto

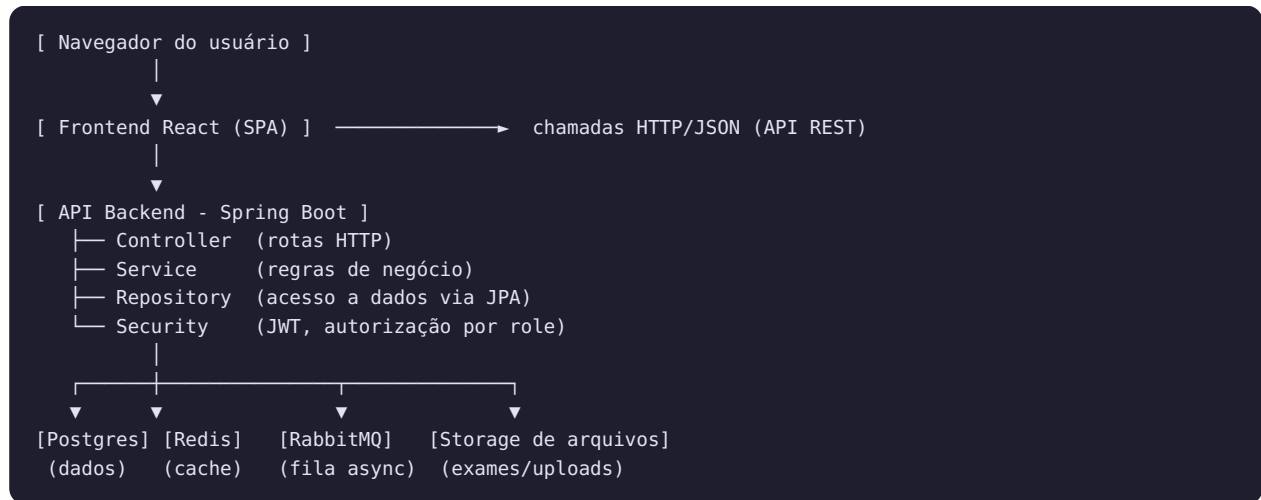
Sobre custos

Tudo neste roadmap roda **100% de graça e localmente** até a Fase 18. A Fase 19 (deploy em nuvem) é opcional e usa camadas gratuitas de provedores (ex.: Oracle Cloud Free Tier, Render, ou Railway) — não é obrigatório pagar nada para completar o projeto inteiro.

VISÃO GERAL

Arquitetura geral do sistema

Como as peças se encaixam, do navegador do usuário até o banco de dados.



Em produção (Fases 14 a 17), cada uma dessas caixas vira um **container Docker independente**, orquestrado pelo Kubernetes, com sua própria escalabilidade e ciclo de vida. É essa separação que permite, por exemplo, reiniciar só o banco sem derrubar a API, ou escalar a API para múltiplas réplicas sem tocar no banco.

Estrutura de pacotes do backend (a ser seguida em todas as fases)

```
com.medflow
├── config          (configurações gerais: segurança, CORS, beans)
├── controller      (rotas REST)
├── dto             (objetos de entrada/saída da API, separados das entidades)
├── entity          (entidades JPA, mapeadas para tabelas)
├── repository      (interfaces JpaRepository)
├── service         (regras de negócio)
├── exception       (exceções customizadas + handler global)
├── security        (filtros JWT, configuração de autenticação)
└── mapper          (conversão entre entity e DTO)
```

Por que separar DTO de Entity

Um **DTO** (Data Transfer Object) é a "forma" dos dados que entram/saem pela API — diferente da **Entity**, que é a forma dos dados no banco. Essa separação evita expor campos sensíveis do banco diretamente na API (ex.: senha) e permite que a API mude de "forma" sem precisar alterar o banco, e vice-versa. Isso será cobrado como padrão em todas as fases de CRUD deste roadmap.

EXECUÇÃO

0 Preparação do ambiente

Assumindo que a máquina não tem absolutamente nada instalado.

Objetivo: deixar a máquina pronta para desenvolver, sem nenhuma dependência do projeto em si ainda.

JDK 21

Maven

Docker Desktop

Git

IntelliJ IDEA

Node.js LTS

Postman/Insomnia

O que instalar

Ferramenta	Para quê
JDK 21	Compilar e rodar o backend Java
Maven	Gerenciar dependências e build do backend (geralmente já embutido na IDE)
IntelliJ IDEA (Community)	IDE principal para o backend
Node.js LTS + npm	Rodar e buildar o frontend React
VS Code	IDE recomendada para o frontend
Docker Desktop	Rodar containers localmente (banco, backend, frontend, etc.)
Git	Controle de versão
Conta no GitHub	Hospedar o repositório e rodar o CI/CD
Postman ou Insomnia	Testar chamadas de API manualmente durante o desenvolvimento
Minikube + kubectl	Rodar Kubernetes localmente (necessário só a partir da Fase 16)

✓ Definição de pronto (DoD) desta fase

- `java -version` mostra Java 21
- `docker --version` e `docker compose version` funcionam
- `git --version` funciona e há uma conta GitHub configurada com SSH ou token
- `node -v` e `npm -v` funcionam
- Um repositório Git vazio chamado `medflow` foi criado no GitHub

EXECUÇÃO

1 Modelagem e planejamento

Antes de escrever qualquer código — desenhar o sistema no papel.

Objetivo: ter um documento (ou conjunto de arquivos) descrevendo entidades, relacionamentos e contratos de API, para que todas as fases seguintes sigam a mesma base, sem retrabalho.

Entregáveis desta fase

- **Diagrama de entidades (DER)** com as tabelas principais: `usuario`, `medico`, `especialidade`, `paciente`, `consulta`, `pagamento`, `prontuario`, `exame`.
- **Lista de casos de uso** por perfil (o que cada role consegue fazer, ligando com o capítulo "O projeto").
- **Rascunho do contrato de API:** lista de endpoints previstos (ex.: `POST /auth/login`, `GET /medicos`, `POST /consultas`), mesmo que ainda não implementados.

Sugestão de entidades e relacionamentos principais

Entidade	Principais campos	Relacionamentos
<code>Usuario</code>	id, nome, email, senha (hash), role	1:1 com Paciente ou Medico, dependendo do role
<code>Especialidade</code>	id, nome, descrição	1:N com Médico
<code>Medico</code>	id, crm, especialidade_id, usuario_id	1:N com Consulta, 1:N com HorarioDisponivel
<code>Paciente</code>	id, cpf, data_nascimento, usuario_id	1:N com Consulta
<code>Consulta</code>	id, medico_id, paciente_id, data_hora, status	1:1 com Pagamento, 1:1 com Prontuario
<code>Pagamento</code>	id, consulta_id, valor, status, forma_pagamento	N:1 com Consulta
<code>Prontuario</code>	id, consulta_id, observacoes	1:N com Exame
<code>Exame</code>	id, prontuario_id, nome_arquivo, url_arquivo	N:1 com Prontuario

💡 Dica de execução com IA

Peça para a IA gerar o DER em formato Mermaid (texto), assim ele fica versionável junto com o código e fácil de visualizar em qualquer lugar (inclusive no próprio GitHub).

✅ Definição de pronto (DoD) desta fase

- Arquivo `docs/modelagem.md` criado no repositório com DER, casos de uso e contrato de API rascunhado
- Todas as entidades da tabela acima estão representadas no DER
- Todos os 4 perfis de usuário têm pelo menos 3 casos de uso listados

EXECUÇÃO

2 Esqueleto do backend

Criar o projeto Spring Boot com a estrutura de pastas definitiva, sem regra de negócio ainda.

Spring Boot 3

Maven

Spring Web

Spring Boot Actuator

O que fazer

- Gerar o projeto via start.spring.io com as dependências: Spring Web, Spring Data JPA, PostgreSQL Driver, Validation, Spring Boot Actuator, Spring Security, Lombok.
- Criar a estrutura de pacotes definida no capítulo "Arquitetura geral" (config, controller, dto, entity, repository, service, exception, security, mapper).
- Configurar o arquivo `application.yml` com perfis separados: `application-dev.yml` e `application-prod.yml`.
- Criar um endpoint simples `GET /health` (ou usar o do Actuator, `/actuator/health`) para validar que o servidor sobe corretamente.
- Configurar um **handler global de exceções** (`@RestControllerAdvice`) desde já, para padronizar erros da API em todo o projeto.

```
# application-dev.yml (exemplo de estrutura esperada)
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/medflow
    username: medflow
    password: medflow
  jpa:
    hibernate:
      ddl-auto: validate
server:
  port: 8080
```

✓ Definição de pronto (DoD) desta fase

- O projeto sobe com `mvn spring-boot:run` sem erros
- `GET /actuator/health` retorna `{"status": "UP"}`
- Estrutura de pacotes criada conforme definido, mesmo que vazia em alguns pontos
- Existe um handler global de exceções retornando um JSON de erro padronizado (ex.: `{"timestamp", "status", "mensagem"}`)

EXECUÇÃO

3

Banco de dados e migrations

O banco de dados entra em cena, versionado desde o primeiro dia.

PostgreSQL

Flyway

Docker Compose (só banco, por enquanto)

O que fazer

- Criar um `docker-compose.yml` inicial **apenas com o PostgreSQL** (os demais serviços entram nas fases seguintes), para não depender de instalar Postgres na máquina.
- Adicionar a dependência do Flyway ao projeto.
- Criar as entidades JPA (`@Entity`) para todas as tabelas definidas na Fase 1.
- Criar os scripts de migration do Flyway (`V1_criar_tabelas_iniciais.sql` , etc.) — **nunca deixar o Hibernate criar/alterar tabelas automaticamente** (`ddl-auto: validate` , nunca `update` ou `create`).

```
# docker-compose.yml (versão inicial, só banco)
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: medflow
      POSTGRES_USER: medflow
      POSTGRES_PASSWORD: medflow
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

⚠ Por que Flyway e não `ddl-auto: update`

Deixar o Hibernate criar/alterar tabelas automaticamente funciona para brincar, mas **nenhuma empresa séria faz isso em produção** — não há histórico do que mudou, não dá para reverter, e é fácil perder dados sem querer. Migrations versionadas (Flyway) são o padrão profissional, e é exatamente esse hábito que você deve treinar aqui.

✅ Definição de pronto (DoD) desta fase

- `docker compose up -d` sobe o Postgres corretamente
- Ao rodar a aplicação, o Flyway aplica as migrations automaticamente e cria todas as tabelas
- Todas as entidades JPA da Fase 1 existem e estão mapeadas para as tabelas corretas
- É possível "derrubar" o volume do banco (`docker compose down -v`) e subir de novo do zero, recriando tudo via migration — esse é o primeiro "derrubar e levantar" do projeto

EXECUÇÃO

4 Autenticação e autorização (JWT)

Antes de qualquer módulo de negócio, o sistema precisa saber quem está pedindo o quê.

Spring Security

JWT

BCrypt

O que fazer

- Endpoint `POST /auth/registrar` : cria um novo usuário (senha sempre salva com hash BCrypt, nunca em texto puro).
- Endpoint `POST /auth/login` : valida credenciais e devolve um token JWT.
- Filtro de autenticação JWT: toda requisição a rotas protegidas deve validar o token do cabeçalho `Authorization: Bearer ...`.
- Configuração de autorização por role: cada rota declara quais roles podem acessá-la (ex.: só `ADMIN` pode listar todos os usuários).
- Endpoint `GET /auth/me` : devolve os dados do usuário autenticado atual, a partir do token.

```
// exemplo de restrição de rota por role
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/usuarios")
public List<UsuarioDTO> listarTodos() { ... }
```

✓ Definição de pronto (DoD) desta fase

- É possível registrar um usuário e fazer login, recebendo um token JWT válido
- Uma rota protegida retorna `401` sem token, e `403` com token de role sem permissão
- Senhas nunca aparecem em texto puro em nenhuma resposta da API nem no banco
- Testado manualmente via Postman/Insomnia com uma coleção salva no repositório

5 Módulo de usuários e perfis

O primeiro CRUD completo do projeto, já usando tudo que foi montado até aqui.

O que fazer

- CRUD completo de usuários (respeitando permissões: um paciente só edita o próprio perfil; admin edita qualquer um).
- Vincular criação de `Paciente` e `Medico` ao cadastro de `Usuario`, conforme o role escolhido no registro.
- Separar claramente DTO de entrada (`CriarUsuarioDTO`), DTO de saída (`UsuarioDTO` , sem senha) e a entidade.
- Validações de campo com Bean Validation (`@NotBlank` , `@Email` , etc.).

✓ Definição de pronto (DoD) desta fase

- CRUD completo funcionando via Postman, com as permissões corretas por role
- Nenhuma senha aparece em nenhuma resposta da API
- Validações retornam erro `400` claro quando campos obrigatórios faltam

6 Módulo de médicos e agenda

Cadastro de especialidades, médicos e seus horários de atendimento.

O que fazer

- CRUD de **Especialidade** (só ADMIN pode criar/editar/remover).
- CRUD de **Medico**, vinculado a uma especialidade e a um usuário com role MEDICO.
- Entidade e endpoints de **HorarioDisponivel**: cada médico define os dias/horários em que atende (ex.: segundas e quartas, 08h-12h).
- Endpoint público (ou para pacientes autenticados) **GET /medicos?especialidade=X** para listar médicos filtrando por especialidade.

✓ Definição de pronto (DoD) desta fase

- É possível cadastrar especialidades, médicos e seus horários disponíveis via API
- É possível listar médicos filtrando por especialidade
- Um médico só consegue editar sua própria agenda, nunca a de outro médico

EXECUÇÃO

7 Módulo de consultas (agendamento)

O coração do sistema — aqui é onde a regra de negócio fica interessante de verdade.

Regras de negócio obrigatórias

- Um paciente não pode agendar uma consulta em um horário fora da disponibilidade cadastrada do médico.
- Um médico não pode ter duas consultas com o mesmo horário (checagem de conflito no banco, não só na aplicação).
- Uma consulta tem um **ciclo de status**: **AGENDADA** → **CONFIRMADA** → **REALIZADA**, ou **AGENDADA** → **CANCELADA**.
- Cancelamento com menos de X horas de antecedência (defina uma regra, ex.: 2h) deve ser bloqueado ou sinalizado de forma diferente.
- Ao criar uma consulta, gerar automaticamente um **Pagamento** vinculado com status **PENDENTE** (integra com a Fase 8).

Endpoints esperados

Rota	Quem acessa	Ação
POST /consultas	Paciente	Agendar nova consulta
GET /consultas/minhas	Paciente/Médico	Listar as próprias consultas
PUT /consultas/{id}/confirmar	Médico	Confirmar consulta agendada
PUT /consultas/{id}/cancelar	Paciente/Recepcionista	Cancelar consulta
PUT /consultas/{id}/realizar	Médico	Marcar consulta como realizada

💡 Ponto de atenção técnico

Peça explicitamente para a IA usar uma **constraint de banco** (índice único composto por médico + data_hora, ou lock otimista) além da validação na aplicação, para realmente garantir que dois agendamentos simultâneos não criem conflito por condição de corrida (race condition).

✅ Definição de pronto (DoD) desta fase

- Não é possível criar duas consultas conflitantes para o mesmo médico, nem tentando "no susto" (requisições simultâneas)
- O ciclo de status funciona conforme descrito, com transições inválidas bloqueadas (ex.: não dá pra "realizar" uma consulta cancelada)
- Um pagamento pendente é criado automaticamente junto com cada consulta

8 Módulo de pagamentos fictícios

Simular um fluxo de cobrança, sem gateway de pagamento real.

O que fazer

- CRUD/consulta de pagamentos vinculados a consultas.
- Endpoint `POST /pagamentos/{id}/pagar` que simula a confirmação de pagamento (muda status para `PAGO`), como se fosse um "webhook" de um gateway fictício.
- Endpoint `POST /pagamentos/{id}/estornar` para simular reembolso (ex.: em caso de cancelamento de consulta).
- Regra: uma consulta só pode ser marcada como `REALIZADA` se o pagamento estiver `PAGO` (decisão de negócio que você pode ajustar).

✓ Definição de pronto (DoD) desta fase

- O fluxo completo pendente → pago → (opcionalmente) estornado funciona e está refletido no status da consulta relacionada
- Existe uma trava impedindo realizar consulta com pagamento pendente

9 Prontuário e exames (upload de arquivos)

Módulo opcional, mas recomendado — treina upload/armazenamento de arquivos.

MultipartFile (Spring)

Armazenamento local em volume Docker

O que fazer

- Endpoint para o médico registrar observações de uma consulta realizada (`Prontuario`).
- Endpoint de upload de arquivo (`multipart/form-data`) para anexar exames a um prontuário — comece salvando em um volume local/Docker; guardar em serviço de nuvem (S3-like) pode ser um extra da Fase 20.
- Validação de tipo/tamanho de arquivo (ex.: só PDF/imagem, até 5MB).

✓ Definição de pronto (DoD) desta fase

- É possível anexar um arquivo a um prontuário e recuperá-lo depois via API
- Arquivos de tipo/tamanho inválido são rejeitados com mensagem clara

10 Notificações assíncronas

Introduzindo processamento em segundo plano com fila de mensagens.

RabbitMQ

Spring AMQP

O que fazer

- Adicionar RabbitMQ ao `docker-compose.yml`.
- Ao agendar, confirmar ou cancelar uma consulta, publicar um evento numa fila (ex.: `fila.notificacoes`).
- Criar um **consumidor** (listener) que processa essa fila e "envia" a notificação — nesta fase, pode ser simulado apenas logando a mensagem formatada como se fosse um e-mail (o objetivo é treinar o fluxo assíncrono, não configurar um servidor de e-mail real).

💡 Por que isso importa

Esse é o primeiro contato do projeto com **processamento assíncrono** — a ideia de que a API responde rápido ao cliente (ex.: "consulta agendada!") sem esperar todo o trabalho secundário (enviar notificação) terminar primeiro. É assim que sistemas reais evitam deixar o usuário esperando por tarefas que não precisam ser instantâneas.

✅ Definição de pronto (DoD) desta fase

- Ao agendar uma consulta, a resposta da API volta imediatamente, sem esperar o "envio" da notificação
- É possível ver, no console do RabbitMQ (interface web) ou nos logs do consumidor, a mensagem sendo processada

11 Testes automatizados

Provar que o sistema continua funcionando conforme ele cresce.

JUnit 5

Mockito

Testcontainers

O que fazer

- **Testes unitários** da camada de `Service` (regras de negócio), usando Mockito para simular os repositórios.
- **Testes de integração** dos endpoints principais, usando **Testcontainers** para subir um Postgres real (em container, descartável) durante os testes — nunca usar banco em memória tipo H2 aqui, para o teste ser fiel ao banco real de produção.
- Cobertura obrigatória mínima: módulo de autenticação e módulo de consultas (as regras mais críticas do sistema).
- Configurar o Maven para rodar todos os testes com `mvn test`.

✓ Definição de pronto (DoD) desta fase

- `mvn test` roda toda a suíte localmente sem erros
- Existe pelo menos um teste garantindo que dois agendamentos conflitantes são rejeitados
- Existe pelo menos um teste garantindo que rotas protegidas bloqueiam usuários sem permissão

12 Documentação da API

Toda API séria é documentada — inclusive para você mesmo consultar depois.

springdoc-openapi

Swagger UI

O que fazer

- Adicionar a dependência `springdoc-openapi-starter-webmvc-ui`.
- Anotar os controllers com descrições (`@Operation` , `@ApiResponse`) nos endpoints principais.
- Validar que a documentação interativa sobe em `/swagger-ui.html` , permitindo testar chamadas direto pelo navegador (inclusive com autenticação JWT configurada na interface).

✓ Definição de pronto (DoD) desta fase

- Todos os endpoints do sistema aparecem documentados em `/swagger-ui.html`
- É possível fazer login e testar uma rota protegida diretamente pela interface do Swagger

13 Frontend em React

Agora sim, a parte visual — consumindo tudo que já foi construído no backend.

React 18

Vite

TypeScript

Tailwind CSS

React Router

Axios

React Query

Telas mínimas a construir

Tela	Perfil	Descrição
Login / Registro	Todos	Autenticação, salvando o token JWT (localStorage ou cookie)
Lista de médicos/especialidades	Paciente	Busca e filtro por especialidade
Agendamento de consulta	Paciente	Seleciona médico, data e horário disponível
Minhas consultas	Paciente/Médico	Lista com status e ações (cancelar, confirmar)
Agenda do médico	Médico	Visualização em calendário/lista das consultas do dia
Dashboard admin	Admin	Gráficos simples de consultas por período, faturamento fictício

Organização recomendada

```
src/
├─ api/           (funções que chamam a API, por módulo)
├─ components/    (componentes reutilizáveis)
├─ pages/         (uma pasta por tela)
├─ hooks/         (hooks customizados, ex: useAuth)
├─ context/       (contexto de autenticação global)
└─ types/         (tipos TypeScript espelhando os DTOs do backend)
```

✓ Definição de pronto (DoD) desta fase

- Um paciente consegue se cadastrar, logar, ver médicos e agendar uma consulta de ponta a ponta pela interface
- Rotas do frontend são protegidas por perfil (ex.: um paciente não acessa a tela de admin)
- Erros da API são exibidos de forma amigável na tela, não como JSON cru

14 Containerização com Docker

O sistema inteiro passa a subir com um único comando.

Docker

Docker Compose

Multi-stage build

O que fazer

- Criar um `Dockerfile` multi-stage para o backend (uma etapa compila com Maven, outra roda apenas o `.jar` final, numa imagem Java enxuta).
- Criar um `Dockerfile` multi-stage para o frontend (uma etapa builda com Node/Vite, outra serve os arquivos estáticos com Nginx).
- Expandir o `docker-compose.yml` para incluir **todos** os serviços: backend, frontend, postgres, redis, rabbitmq.
- Configurar variáveis de ambiente via `.env` (nunca senhas hardcoded no compose).

```
# Dockerfile backend (multi-stage, exemplo)
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

⚠ Este é o momento de "derrubar e levantar" de verdade

A partir daqui, pratique isso sem dó: `docker compose down -v` (derruba tudo, inclusive dados) e `docker compose up -d --build` (sobe tudo de novo do zero). Esse ciclo é exatamente o que você pediu para treinar — e é assim que times reais testam se o ambiente é realmente reproduzível.

✅ Definição de pronto (DoD) desta fase

- `docker compose up -d --build` sobe o sistema inteiro (backend, frontend, banco, cache, fila) com um único comando, em uma máquina limpa
- O frontend, servido via Nginx, consegue se comunicar com o backend dentro da rede Docker
- `docker compose down -v` seguido de `up` novamente recria tudo do zero sem intervenção manual

15 CI/CD com GitHub Actions

Automatizar build, testes e geração de imagens Docker a cada push.

[GitHub Actions](#)[GitHub Container Registry \(ghcr.io\)](#)

O que fazer

- Criar workflow `.github/workflows/ci.yml` que, a cada push/PR: builda o backend, roda os testes (Fase 11), builda o frontend.
- Criar workflow separado `.github/workflows/docker-publish.yml` que, ao fazer merge na branch principal, builda as imagens Docker e publica no GitHub Container Registry (gratuito).
- Adicionar badge de status do CI no `README.md` do projeto.

```
# .github/workflows/ci.yml (estrutura esperada)
name: CI
on: [push, pull_request]
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: mvn -B test --file backend/pom.xml
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci && npm run build
        working-directory: frontend
```

✓ Definição de pronto (DoD) desta fase

- Todo push dispara o pipeline automaticamente, visível na aba "Actions" do GitHub
- Um Pull Request com testes quebrando é bloqueado/sinalizado automaticamente
- Após merge na branch principal, as imagens Docker são publicadas automaticamente

16 Orquestração com Kubernetes

Simular um ambiente de produção real, com Kubernetes rodando localmente.

Minikube

kubectl

Deployments

Services

ConfigMaps & Secrets

Ingress

O que fazer

- Instalar e iniciar o Minikube localmente (`minikube start`).
- Criar manifests YAML na pasta `k8s/` para cada serviço: `Deployment` + `Service` para backend, frontend, Postgres, Redis e RabbitMQ.
- Usar `ConfigMap` para configurações não sensíveis (ex.: URL do banco) e `Secret` para dados sensíveis (senhas, chave JWT).
- Configurar um `PersistentVolumeClaim` para os dados do Postgres não se perderem ao reiniciar o pod.
- Configurar um `Ingress` (ou `Service` tipo `NodePort` / `LoadBalancer` localmente) para expor o frontend e a API para fora do cluster.
- **(Opcional avançado)** Empacotar tudo isso como um **Helm chart**, para instalar o sistema inteiro com um único comando `helm install`.

```
# exemplo simplificado de Deployment do backend
apiVersion: apps/v1
kind: Deployment
metadata:
  name: medflow-backend
spec:
  replicas: 2
  selector:
    matchLabels: { app: medflow-backend }
  template:
    metadata:
      labels: { app: medflow-backend }
    spec:
      containers:
        - name: backend
          image: ghcr.io/SEU_USUARIO/medflow-backend:latest
          ports: [{ containerPort: 8080 }]
          envFrom:
            - configMapRef: { name: medflow-config }
            - secretRef: { name: medflow-secrets }
```

💡 O playground que você pediu

É exatamente aqui que "derrubar o banco e levantar de novo" vira ainda mais interessante: experimente deletar o pod do Postgres (`kubectl delete pod postgres-xxxx`) e observe o Kubernetes recriá-lo automaticamente; teste escalar o backend (`kubectl scale deployment medflow-backend --replicas=4`) e veja múltiplas réplicas dividindo a carga.

✅ Definição de pronto (DoD) desta fase

- O sistema inteiro sobe no Minikube com `kubectl apply -f k8s/`
- O frontend é acessível via navegador através do Ingress/Service exposto
- Deletar um pod do backend faz o Kubernetes recriá-lo automaticamente (self-healing)
- Dados do Postgres sobrevivem a um `kubectl delete pod` (graças ao `PersistentVolumeClaim`)

17 Observabilidade

Saber o que está acontecendo dentro do sistema, sem precisar adivinhar.

[Spring Boot Actuator](#)[Micrometer](#)[Prometheus](#)[Grafana](#)

O que fazer

- Expor métricas do backend via Actuator + Micrometer no formato que o Prometheus entende (`/actuator/prometheus`).
- Adicionar Prometheus e Grafana como serviços no Kubernetes (ou no Docker Compose, para uma versão mais simples).
- Configurar o Prometheus para "raspar" (scrape) as métricas do backend periodicamente.
- Montar um dashboard básico no Grafana: número de requisições por rota, tempo de resposta médio, uso de memória/CPU dos pods.
- Padronizar logs em formato JSON estruturado, incluindo um `traceId` por requisição para facilitar depuração.

✓ Definição de pronto (DoD) desta fase

- O Grafana exibe um dashboard funcional com métricas reais do backend rodando
- É possível identificar, pelo dashboard, um pico de requisições ou erro em tempo real durante um teste manual

18 Segurança e hardening

Fechando as portas que ficaram abertas "só para desenvolver mais rápido".

Checklist de segurança a aplicar

- ✓ CORS configurado explicitamente (não usar `*` em produção)
- ✓ Rate limiting básico nas rotas de login (evitar força bruta)
- ✓ Segredos (JWT secret, senha do banco) fora do código-fonte, via variáveis de ambiente/Secrets do Kubernetes
- ✓ Validação de entrada em todos os endpoints (Bean Validation já usado desde a Fase 5, revisar cobertura)
- ✓ Headers de segurança HTTP básicos configurados (ex.: via Spring Security)
- ✓ Dependências do projeto escaneadas por vulnerabilidades conhecidas (ex.: `mvn dependency-check` ou Dependabot do GitHub)
- ✓ Usuário do container Docker rodando sem privilégios de root

✓ Definição de pronto (DoD) desta fase

- Todos os itens do checklist acima foram revisados e aplicados
- Dependabot (ou equivalente) está ativo no repositório GitHub

19 Deploy em nuvem (opcional)

Tirar o projeto da sua máquina e colocá-lo acessível de verdade na internet.

Opções gratuitas/baratas para considerar

Serviço	Uso
Oracle Cloud Free Tier	Máquina virtual gratuita "para sempre", boa para rodar Docker Compose direto
Railway / Render	Deploy simplificado de containers, camada gratuita limitada
Google Kubernetes Engine (GKE) / AWS EKS	Kubernetes gerenciado de verdade, ótimo para aprender, mas geralmente pago (fora créditos gratuitos iniciais)

Sugestão de caminho

Comece publicando a versão Docker Compose (Fase 14) numa VM gratuita — isso já é "produção de verdade", com endereço IP público acessível. Só depois, se quiser ir além, migre para um Kubernetes gerenciado usando os manifests já prontos da Fase 16.

Definição de pronto (DoD) desta fase

- O sistema está acessível por uma URL pública, fora da sua máquina local
- HTTPS configurado (ex.: via Let's Encrypt/Certbot ou o provedor escolhido)

20 Extras / próximos níveis

Depois que o núcleo estiver sólido, ideias para continuar evoluindo o projeto.

- **Multi-tenancy:** transformar o MedFlow para atender várias clínicas diferentes no mesmo sistema, isoladas entre si.
- **Armazenamento de arquivos em nuvem:** trocar o armazenamento local de exames (Fase 9) por um serviço S3-compatível (ex.: MinIO, rodando também em container).
- **Autoscaling real:** configurar `HorizontalPodAutoscaler` no Kubernetes, escalando o backend automaticamente conforme o uso de CPU.
- **Blue-green ou canary deploy:** evoluir o CI/CD para atualizar a aplicação em produção sem downtime.
- **Cache avançado:** usar Redis não só como cache simples, mas para invalidar dados automaticamente quando um médico ou especialidade é atualizado.
- **Relatórios avançados:** exportação de relatórios do dashboard em PDF/Excel.
- **App mobile:** consumir a mesma API REST com React Native ou Flutter.



Pensando à frente

Esses extras não são obrigatórios — são o tipo de coisa que times reais adicionam com o tempo, conforme o sistema cresce. Ter esse "backlog" já pronto ajuda a manter o projeto vivo depois que o roadmap principal estiver concluído, em vez de ele "morrer" assim que as 20 fases acabarem.

Anexo A — Modelo de prompt para cada fase

Copie, adapte o número da fase e cole numa IA de codificação.

Modelo de prompt

Você vai me ajudar a implementar a Fase [NÚMERO] do projeto MedFlow, um sistema de gestão de clínica médica fictícia.

Contexto fixo do projeto (não mude estas decisões sem me avisar):

- Backend: Java 21 + Spring Boot 3, Maven, PostgreSQL + Flyway, Spring Security + JWT
- Frontend: React + Vite + TypeScript + Tailwind
- Infra: Docker/Docker Compose, Kubernetes (Minikube), GitHub Actions
- Arquitetura em camadas: controller / service / repository / dto / entity / mapper
- Já implementado até agora: [resuma rapidamente as fases anteriores já feitas]

Objetivo desta fase:

[cole aqui o conteúdo da fase correspondente deste roadmap: objetivo, o que fazer, entregáveis]

Definição de pronto (DoD) que preciso validar ao final:

[cole aqui os itens do "Definição de pronto" desta fase]

Regras:

1. Implemente apenas o escopo desta fase, não adiante fases futuras.
2. Explique brevemente cada decisão técnica não óbvia que você tomar.
3. Ao final, me diga exatamente quais comandos rodar para testar que essa fase está funcionando.
4. Se algo do roadmap estiver ambíguo, pergunte antes de assumir.

Dica extra

Depois que a IA implementar uma fase, rode você mesmo os testes/comandos sugeridos antes de avançar. Esse é o momento de realmente entender o que foi feito — pergunte "por que você fez assim?" sempre que algo não fizer sentido pra você.

Anexo B — Checklist geral de entrega

Uma visão rápida de todas as fases, para marcar o progresso do projeto inteiro.

- ✓ Fase 0 — Ambiente instalado e repositório criado
- ✓ Fase 1 — Modelagem documentada (DER, casos de uso, contrato de API)
- ✓ Fase 2 — Esqueleto do backend rodando
- ✓ Fase 3 — Banco de dados versionado com Flyway
- ✓ Fase 4 — Login/JWT funcionando com autorização por role
- ✓ Fase 5 — CRUD de usuários completo
- ✓ Fase 6 — Médicos, especialidades e agenda cadastrados
- ✓ Fase 7 — Agendamento de consultas com regras de conflito
- ✓ Fase 8 — Fluxo de pagamento fictício completo
- ✓ Fase 9 — Upload de exames funcionando
- ✓ Fase 10 — Notificações assíncronas via fila
- ✓ Fase 11 — Suíte de testes automatizados passando
- ✓ Fase 12 — Documentação Swagger completa
- ✓ Fase 13 — Frontend completo e integrado
- ✓ Fase 14 — Sistema inteiro sobe via Docker Compose
- ✓ Fase 15 — Pipeline de CI/CD funcionando
- ✓ Fase 16 — Sistema rodando em Kubernetes local
- ✓ Fase 17 — Dashboard de observabilidade funcionando
- ✓ Fase 18 — Checklist de segurança revisado
- ✓ Fase 19 — Deploy público (opcional) no ar
- ✓ Fase 20 — Ao menos um extra implementado (opcional)

Última dica

Não existe problema nenhum em ficar travado numa fase por dias — é aí que o aprendizado de verdade acontece. O objetivo deste roadmap não é terminar rápido, é terminar entendendo cada camada do sistema o suficiente para conseguir explicá-la para outra pessoa.